

Tutorial: Phase 2 - Backend API Development

Introduction

This tutorial provides step-by-step instructions for developing the **PHP-based REST API** for the **Pandemic Resilience System (PRS)**. The API will support user management, vaccination record handling, document uploads, and audit logging.

Objectives:

Develop PHP-based REST API for:

- User management (CRUD operations)
- Vaccination records retrieval & insertion
- Secure document uploads
- Audit logging : Implement authentication & role-based access control ,
Secure API endpoints with HTTPS & input validation

Deliverables:

- API documentation (Postman collection)
- PHP API endpoints hosted locally
- Security testing on API (SQL Injection, CSRF, etc.)

Step 1: Set Up API Environment

Install Required Software

Ensure you have the following installed:

- **PHP** (≥ 7.4)
- **MySQL**
- **Apache/Nginx** (via XAMPP/LAMP)
- **Postman** (for API testing)

◆ Configure Apache for API Support

1. Enable **mod_rewrite** for URL routing.
2. Update **.htaccess** file to allow clean API URLs:
3. RewriteEngine On

4. RewriteCond %{REQUEST_FILENAME} !-f
5. RewriteRule ^ api.php [QSA,L]

Step 2: Develop API Endpoints

Database Connection (db.php)

```
<?php
$host = 'localhost';
$dbname = 'prs_database';
$username = 'root';
$password = '';
$conn = new mysqli($host, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

API Routing (api.php)

```
<?php
require 'db.php';
header('Content-Type: application/json');

$method = $_SERVER['REQUEST_METHOD'];
$request = explode('/', trim($_SERVER['PATH_INFO'], '/'));

if ($method === 'GET' && $request[0] === 'users') {
    $result = $conn->query("SELECT * FROM users");
    echo json_encode($result->fetch_all(MYSQLI_ASSOC));
}
?>
```

Create User API

```
if ($method === 'POST' && $request[0] === 'users') {
    $data = json_decode(file_get_contents("php://input"), true);
    $stmt = $conn->prepare("INSERT INTO users (full_name, email, password_hash, role_id) VALUES (?, ?, ?, ?)");
    $stmt->bind_param("sssi", $data['full_name'], $data['email'], password_hash($data['password'], PASSWORD_DEFAULT), $data['role_id']);
```

```

        echo json_encode(["status" => $stmt->execute() ? "success" :
"error"]);
    }
}

```

Step 3: Implement Authentication & Role-Based Access

User Login & JWT Authentication

```
require 'jwt_handler.php';
```

```

if ($method === 'POST' && $request[0] === 'login') {
    $data = json_decode(file_get_contents("php://input"), true);
    $stmt = $conn->prepare("SELECT user_id, password_hash FROM users
WHERE email = ?");
    $stmt->bind_param("s", $data['email']);
    $stmt->execute();
    $stmt->bind_result($user_id, $password_hash);
    $stmt->fetch();
    if (password_verify($data['password'], $password_hash)) {
        echo json_encode(["token" => createJWT($user_id)]);
    } else {
        echo json_encode(["error" => "Invalid credentials"]);
    }
}
}

```

Secure API Endpoints with JWT Middleware

```

function authenticate() {
    $headers = getallheaders();
    if (!isset($headers['Authorization']) ||
!validateJWT($headers['Authorization'])) {
        http_response_code(403);
        die(json_encode(["error" => "Access Denied"]));
    }
}

```

Step 4: Secure Document Uploads

```

if ($method === 'POST' && $request[0] === 'upload') {
    authenticate();
    if (isset($_FILES['file'])) {

```

```

        $file_name = basename($_FILES['file']['name']);
        move_uploaded_file($_FILES['file']['tmp_name'], "uploads/" .
$file_name);
        echo json_encode(["status" => "success", "file_path" =>
"uploads/$file_name"]);
    }
}

```

Step 5: Security Testing

SQL Injection Protection

- Use **prepared statements** (\$stmt->bind_param()).
- Reject malformed SQL inputs.

CSRF Protection

- Generate **CSRF tokens** for sensitive actions.
- Require token validation on **form submissions**.

API Security Hardening

- **Enforce HTTPS** for all requests.
- **Limit failed login attempts**.
- **Validate JWT expiration times**.

Deliverables

- API documentation (Postman collection)
- Fully working PHP API with authentication & security
- Security audit report (SQL Injection, CSRF tests)